

CreditPathAI – Milestone 4: Advanced Model Training (XGBoost & LightGBM)

Objective

Train high-performance models (XGBoost and LightGBM), perform hyperparameter tuning, and improve AUC-ROC compared to the Logistic Regression baseline.

Why Advanced Models

Logistic Regression is linear and cannot capture complex patterns. XGBoost and LightGBM: • Capture non-linear relationships • Automatically learn feature interactions • Perform well on structured/tabular data
Alternative: Random Forest Drawback: slower and less optimized than boosting methods.

Step 1: Load Data

```
df = pd.read_csv("final_features.csv")
```

 We use engineered features, not raw data.

Step 2: Train-Test Split

```
from sklearn.model_selection import train_test_split
X = df.drop("default_status", axis=1)
y = df["default_status"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

 Reason: prevents overfitting.

Step 3: Train XGBoost

```
from xgboost import XGBClassifier
xgb = XGBClassifier(n_estimators=100, max_depth=5, learning_rate=0.1, subsample=0.8, colsample_bytree=0.8, random_state=42)
xgb.fit(X_train, y_train)
```

 Each parameter controls model complexity and generalization.

Step 4: Train LightGBM

```
from lightgbm import LGBMClassifier
lgbm = LGBMClassifier(n_estimators=100, max_depth=5, learning_rate=0.1, subsample=0.8, colsample_bytree=0.8, random_state=42)
lgbm.fit(X_train, y_train)
```

 LightGBM is faster and uses leaf-wise growth.

Step 5: Predictions

`y_prob_xgb = xgb.predict_proba(X_test)[:,-1]` `y_prob_lgbm = lgbm.predict_proba(X_test)[:,-1]` We use probabilities for AUC calculation.

Step 6: AUC Evaluation

`from sklearn.metrics import roc_auc_score` `auc_xgb = roc_auc_score(y_test, y_prob_xgb)` `auc_lgbm = roc_auc_score(y_test, y_prob_lgbm)` Goal: outperform Logistic Regression baseline.

Step 7: Hyperparameter Tuning

`from sklearn.model_selection import GridSearchCV` `param_grid = { 'max_depth':[3,5,7], 'learning_rate':[0.01,0.1], 'n_estimators':[100,200] }` `grid = GridSearchCV(XGBClassifier(), param_grid, scoring='roc_auc', cv=3)` `grid.fit(X_train, y_train)` `best_model = grid.best_estimator_` Systematic tuning improves performance.

Step 8: Feature Importance

`importance = pd.DataFrame({ "Feature": X.columns, "Importance": xgb.feature_importances_ }).sort_values(by="Importance", ascending=False)` Helps identify most impactful features.

Final Evaluation

Compare AUC scores: Logistic Regression vs XGBoost vs LightGBM Expected: Advanced models should perform better. If not → improve feature engineering.